

DEV_IMPL_GUIDE - TCK-01 Ticketing and Case Management

Use with DD_TCK-01-Ticketing_and_Case_Management-v1.0.md.

This guide explains how the main ticketing flows execute at runtime.

1. Runtime Components

Component	Responsibility
TCK API	Receives ticket commands and queries.
TCK DB	Stores ticket, links, comments, attachments, timeline.
TCK outbox dispatcher	Publishes ticket events.
TCK inbox worker	Consumes WO events.
WO client	Creates Work Orders and reads WO state.
ILM/SUB/BIL/OSR clients	Load linked entity summaries.
NOT/ICN client	Sends customer/staff notifications.

2. Flow - Create Ticket From Backoffice

Request:

```
POST /api/tickets
Idempotency-Key: ui-req-01J2SUPPORT001
Authorization: Bearer <jwt>
Content-Type: application/json

{
  "operatorCode": "WIK",
  "customerId": "cust_01HZ_KAMAU",
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "typeCode": "TECHNICAL_SUPPORT",
  "categoryCode": "NO_INTERNET",
  "sourceChannel": "CALL_CENTER",
  "subject": "Customer has no internet",
  "description": "Customer reports LOS red light on ONT."
}
```

Execution:

1. TCK API authenticates user and checks role.
2. TCK checks idempotency key before inserting.
3. TCK validates category from ticket_category_catalog.
4. TCK loads customer from ILM API.
5. If subscriptionId is present, TCK loads subscription from SUB-LM API.
6. TCK computes priority, queue, and SLA due dates from category config.
7. In one DB transaction:
 - insert ticket;
 - insert ticket_link rows;
 - insert ticket_timeline_event TicketCreated;
 - insert outbox event TicketCreated.
8. TCK returns ticket summary.
9. Outbox dispatcher publishes TicketCreated.

Do not call WO during simple ticket creation unless the API request is specifically create work order from ticket.

3. Flow - Create Work Order From Ticket

Request:

```
POST /api/tickets/tck_01J2SUPPORT001/work-orders
Idempotency-Key: ui-req-01J2W0001
Authorization: Bearer <jwt>
Content-Type: application/json

{
  "woKind": "SUPPORT",
  "taskType": "SUPPORT",
  "preferredDate": "2026-06-02",
  "techRegionCode": "NRB-KAREN",
  "description": "No internet. Check ONT LOS and fiber drop.",
  "equipmentRefs": ["eqp_ONT_42"]
}
```

Execution:

1. TCK locks ticket row with FOR UPDATE.
2. Validate ticket is not terminal.
3. Validate category has wo_allowed = true.
4. Validate no active linked WO exists unless category allows another WO.
5. Load customer/account from ILM if needed for WO payload.
6. Load subscription from SUB-LM if ticket is subscription-linked.
7. Validate equipment refs through OSR if supplied.
8. Build WO request with callerContext.
9. Call WO-01 POST /api/work-orders.
10. In a TCK DB transaction:

- set ticket status_code = WAITING_WORK_ORDER;
- set current_wo_id;
- insert ticket_link for the WO;
- insert timeline event WorkOrderCreatedFromTicket;
- insert outbox event TicketWorkOrderCreated.

11. Return ticket + WO summary.

If WO creation fails before a WO id is returned, leave ticket unchanged and return controlled error.

If WO creation returns success but TCK DB update fails, use a reconciliation job:

1. Search WO by callerContext.callerTicketId.
2. Link missing WO back to ticket.
3. Add timeline event.

4. WO Request Example

TCK sends:

```
POST /api/work-orders
Authorization: Bearer <service-token>
Content-Type: application/json

{
  "operatorCode": "WIK",
  "kind": "SUPPORT",
  "taskType": "SUPPORT",
  "customerId": "cust_01HZ_KAMAU",
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "techRegionCode": "NRB-KAREN",
  "description": "No internet. Check ONT LOS and fiber drop.",
  "callerContext": {
    "callerModule": "TCK",
    "callerTicketId": "tck_01J2SUPPORT001",
    "callerTicketNumber": "KE-TCK-2026-000001"
  },
  "equipmentRefs": ["eqp_ONT_42"]
}
```

```
}
```

5. Flow - Consume WorkOrderFinalized

Kafka event arrives:

```
{
  "eventType": "WorkOrderFinalized",
  "aggregateId": "wo_01HZ_SUPPORT_01",
  "payload": {
    "callerContext": {
      "callerModule": "TCK",
      "callerTicketId": "tck_01J2SUPPORT001"
    },
    "finalOutcomeCode": "RESOLVED_ONSITE",
    "summary": "Damaged drop cable replaced."
  }
}
```

Execution:

1. Kafka consumer writes event into inbox table.
2. Inbox worker ignores event if already processed.
3. If callerModule != TCK, mark no-op processed.
4. Load ticket by callerTicketId.
5. Insert timeline event LinkedWorkOrderFinalized.
6. If category config auto-resolves on successful WO outcome:
 - set ticket status_code = RESOLVED;
 - set resolution fields.
7. Else set ticket status_code = UNDER_REVIEW.
8. Insert outbox event TicketResolved or TicketUpdated.
9. Mark inbox processed.

6. Flow - Resolve Ticket

Request:

```
POST /api/tickets/tck_01J2SUPPORT001/resolve
Content-Type: application/json

{
  "resolutionCode": "SERVICE_RESTORED",
  "resolutionSummary": "Technician replaced damaged drop cable. Service restored.",
  "customerVisible": true
}
```

Execution:

1. Lock ticket.
2. Validate status is not terminal.
3. Validate no active linked WO is still in progress.
4. Validate resolution code is allowed for category.
5. Set status RESOLVED, resolution fields, resolved_at.
6. Insert timeline event TicketResolved.
7. Insert outbox event TicketResolved.
8. If customer visible, call NOT/ICN through event or approved notification API.

7. Required Tests

- Duplicate ticket create with same idempotency key and same hash returns existing ticket.

- Same idempotency key with different hash returns 409.
- Category not WO-enabled rejects create-WO-from-ticket.
- Terminal ticket rejects create-WO-from-ticket.
- Active linked WO rejects second WO creation.
- WO creation success links WO and changes status to `WAITING_WORK_ORDER`.
- `WorkOrderFinalized` event moves ticket to `UNDER_REVIEW` or `RESOLVED`.
- Duplicate WO event is ignored by inbox dedup.
- Ticket resolution fails if linked WO is still active.

8. Developer Notes

- Keep ticket links generic. Do not add nullable columns for every possible linked entity.
- Keep customer-visible and internal comments separated.
- Keep all status changes in timeline.
- Treat WO as the owner of field execution. TCK tracks why the WO exists and what customer issue it resolves.
- Do not make Customer 360 call WO directly for ticket-linked status if TCK already has the ticket timeline; use TCK for case history and WO for WO detail.